MASTER'S THESIS DEFENCE

When does better locality make memory slower?

DRAM locality in graph-based approximate nearest neighbour search

Artashes Atanesyan

10 November 2026

Question: if graph search needs the same data, can changing its placement and the order of available reads finish memory service sooner?

Row reuse saves operations. Bank parallelism can hide waiting.

We follow this tension from controlled traces to HNSW graph search, then to a proposed processing-in-memory (PIM) interface.

Evidence: Full thesis; simulated DRAM service, not measured application/PIM speedup. Links open the maintained artifact; pin a Git revision for reproduction.

Established methods; new evidence for their use

RQ1 — Access order
How do order, runs and presorting interact with request concurrency?

RQ2 — Graph-search choices
Which placements and available-read schedules help? Can training select one?

RQ3 — Robustness
Which conclusions survive the selected footprint, boundary, controller and graph changes?

Established prior work
Gorder/Porder: measured placements.
PIM systems: literature comparisons.
Locality and scheduling are established.

This thesis adds
C1: placement failure cases.
C2: scheduling reversals, explained.
C3: implementation cost thresholds.

Use: test a fetch order's assumptions and cost before integration.

Evidence: Research position and cited prior work; selected references on slide 14. Links open the maintained artifact; pin a Git revision for reproduction.

Change addresses and order; preserve search work

Search Discover candidate IDs → **Placement** Map IDs to vector addresses → **Request order** Offer native memory reads → **Controller** Measure completed service cycles

Workload

HNSW: graph-based approximate nearest neighbours; SIFT100k, 128 dimensions.
Ten paired splits; 200 held-out queries each. Mean Recall@10: 98.99%.

Memory model

Pinned Ramulator 2.1, one HBM2 channel.
512-byte float32 vector: 16 native reads.
Hypothetical 128-byte vector: 4 reads,
with the same float32 traversal and recall.

Evidence: Fixed traces, stock-search comparisons, native 32-byte accounting and paired protocol. Links open the maintained artifact; pin a Git revision for reproduction.

Execution boundaries change available parallelism

Flat replay	List → v1, v2, v3, v4 → next list
Expansion phases	List | v1, v2, v3, v4 | next list
Batches of two	List | v1, v2 | v3, v4 | next list

Each divider means: finish every read in this group before offering the next.

List = neighbour IDs. v1–v4 = vectors needed by those candidates.

Flat replay offers known future demand early. **Bounded replay** waits for memory groups, but executes no distance computation or control logic.

Evidence: Replay models; list = neighbour IDs, v1–v4 = candidate vectors; rows stay warm across drains. Links open the maintained artifact; pin a Git revision for reproduction.

A measurable effect can have little practical value

Query-aware placement versus breadth-first (BFS) placement

Default flat replay, same search work; ten paired split effects.

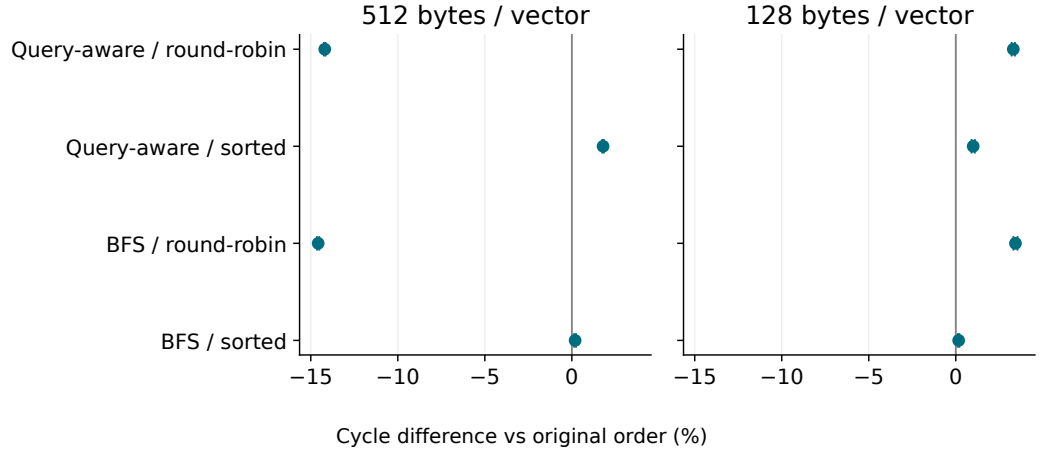
Vector footprint	Mean cycle change	Paired 95% interval
512 bytes	-0.31%	[-0.37, -0.26]%
128 bytes (hypothetical)	+0.17%	[+0.11, +0.24]%

- **Statistical:** the conditional intervals exclude zero.
- **Practical:** only about 97.3 cycles/query saved at 512 bytes, before layout and integration costs.
- **Architectural:** the 128-byte layout has more row hits yet takes longer. Row-hit fraction alone cannot choose a layout.

Evidence: Generated primary placement table; published Gorder/Porder comparisons retained separately. Links open the maintained artifact; pin a Git revision for reproduction.

Round-robin helps one footprint and hurts the other

Round-robin interleaves chunks of available vectors; original order offers each vector's chunks together.



BFS: -14.58% at 512 bytes; +3.43% at hypothetical 128 bytes.

Evidence: Expansion-bounded schedules, ten paired splits; bars = 95% intervals; negative = fewer cycles. Links open the maintained artifact; pin a Git revision for reproduction.

More row operations, much less waiting

Reference split 1: expansion-bounded BFS, 512 bytes, round-robin versus original.

Diagnostic	Original	Round-robin
Row-hit fraction	93.41%	89.23%
Offered bank diversity	4.75	15.07

$$\underbrace{+277{,}894}_{\text{command cycles}} \quad - \quad \underbrace{1{,}429{,}356}_{\text{no-command cycles}} \quad = \quad \underbrace{-1{,}151{,}462}_{\text{total cycles}}$$

The additional row work is outweighed by less idle service time.

Bank diversity counts distinct banks in 64-request input windows, not live queue occupancy. No-command cycles include several causes; this is one diagnostic split, not a new statistical replication.

Evidence: Generated command accounting; the same mechanism does not guarantee a gain at 128 bytes. Links open the maintained artifact; pin a Git revision for reproduction.

Footprint alone cannot select a schedule

Round-robin vs. matching original BFS	512 bytes	128 bytes*
Default: 64 outstanding	-14.58%	+3.43%
16 outstanding	-33.08%	-10.61%
32 outstanding	-21.86%	+1.62%
128 outstanding	-14.58%	+3.43%
Candidate batches of 2	-7.77%	-1.48%
Candidate batches of 8	-15.55%	-2.48%
Alternative address mapping	+112.71%	+42.59%

The same order can help or hurt as execution constraints change.

Ten paired split means. Only the named setting changes. *Hypothetical footprint.
The 128-outstanding cap is nonbinding: all 60 records match default cycles.

Evidence: Complete sensitivity matrix with paired intervals, sorting, absolute savings and query effects. Links open the maintained artifact; pin a Git revision for reproduction.

Profile once, then use a fixed order

50 training queries × 3 orders ⟶ freeze the choice ⟶ 200 held-out queries

Observed

140/140 choices match the best of the three fixed held-out orders.

7 settings × 2 footprints × 10 splits, on one graph and query pool.

Original / sorted / round-robin:
40 / 10 / 90 choices.

Interpretation

Exploratory: inspected test outcomes motivated the policy. No universal success rate or benefit over a correctly chosen fixed schedule is established.

Default 512-byte selected BFS vs. Porder: -14.25 [-14.36, -14.14]%.

At 128 bytes vs. Gorder: +1.35 [+1.26, +1.44]%.

Individual-query losses remain.

Evidence: Training-only choice rule, fixed alternatives, published baselines, costs and limitations. Links open the maintained artifact; pin a Git revision for reproduction.

The implementation has an overhead budget

Default bounded BFS, 512 bytes

About 5,802 service cycles saved per query before extra costs.
Mean calibration: 5,716,851 cycles.
Calibration / saving ≈ 985 queries.
If extra work consumes half the saving,
the ratio becomes about 1,971 queries.

What can erase it?

Additional dispatch, address mapping, buffering, synchronisation and transfers. Charge incremental costs when comparing schedules on the same engine.
The 200 held-out queries save only
1.16 million service cycles, less than
5.72 million spent on calibration.

A correct fixed schedule earns the same replay benefit without repeating calibration.

Ratios of retained means under stable, serial additive costs; not measured deployment break-even. Workload lifetime, overlap, drift and real engine costs remain unknown.

Evidence: Worked amortisation example; raw selection and cost summaries retained. Links open the maintained artifact; pin a Git revision for reproduction.

A concrete interface to evaluate next

Proposed, not implemented: keep vectors resident and return tagged distances.

Host control
Read neighbour IDs from DRAM.
Maintain queues and visited state.
Resolve candidate vector addresses.

Near-memory engine
Retain query in local storage.
Gather resident vectors in chosen order.
Compute and tag their distances.

$$\text{Host} \xrightarrow{\text{query once; then addresses + tags}} \text{engine}$$

$$\text{Host} \xleftarrow{\text{distances + tags}} \text{engine}$$

Requires local arithmetic/storage, tagged fine-grained requests and compatible mapping. Preserve numerical, tie and search-update semantics. Whole-vector or bank-local interfaces may not expose the tested ordering.

DRAM scheduling guides this interface; complete PIM performance remains unmeasured.

Evidence: Residency, transferred data, hardware requirements and unmeasured execution costs. Links open the maintained artifact; pin a Git revision for reproduction.

Choose with completed service, then check full cost

RQ1: locality and concurrency interact. Access order can reverse a ranking; neither row hits nor contiguous runs alone establish lower time.

RQ2: named placements and schedules provide conditional alternatives. Profiling selects useful fixed orders in this exploratory matrix.

RQ3: several rankings reverse under the tested changes. The evidence does not establish a universal schedule or footprint rule.

Footprint + available work + mapping + boundaries determine the useful memory schedule.

Delivered: reproducible failure cases, useful alternatives and cost thresholds.
Still required for PIM speedup: implemented compute, buffers, transfers, scheduling costs, and end-to-end validation at equal recall and concurrency.

Evidence: Current PDFs and validation; 2,069 records = 1,289 previous + 360 sensitivity + 420 calibration. Links open the maintained artifact; pin a Git revision for reproduction.

What the evidence can and cannot answer

- **Why ten splits?** They quantify paired query variation on one graph. Overlapping pools and exploratory comparisons limit inference; they do not establish hardware or dataset generality.
- **Why 128 bytes?** A controlled traffic/packing sensitivity with float32 search held fixed. Quantisation accuracy and arithmetic were not evaluated.
- **Why HNSW/SIFT100k?** A reproducible graph-search operating point; one real corpus, one main parameter/recall setting. Three additional graph constructions test limited variation.
- **What is validated?** Retained accounting, membership, source identities, analysis regeneration and three separate fresh simulations. The simulator has not been validated against physical PIM hardware here.

Evidence: Thirteen expanded defence answers; reproduction inventory and historical-baseline distinction. Links open the maintained artifact; pin a Git revision for reproduction.

Established methods frame the contribution

Graph search. Malkov and Yashunin, HNSW, IEEE TPAMI 42(4), 2020.
doi:10.1109/TPAMI.2018.2889473

Placement. Wei et al., Gorder, SIGMOD 2016, pp. 1813–1828.
doi:10.1145/2882903.2915220. Coleman et al., graph reordering / Porder, NeurIPS 35, 2022.
doi:10.52202/068431-2789

DRAM model. Luo et al., Ramulator 2.0, IEEE CAL 23(1), 2024.
doi:10.1109/LCA.2023.3333759. Experiments pin the later Ramulator 2.1 source.

PIM systems. DRIM-ANN and UpANNS, SC 2025; PIMANN, USENIX ATC 2025.
PIMCQG, arXiv:2605.25522v2; public record states SC26 acceptance (checked 10 Nov. 2026).

Full citations, publication status and the architectural distinctions are in the thesis bibliography and related-work section.

Evidence: Shared thesis/IEEE bibliography; talk guidance in READING_GUIDE.md. Links open the maintained artifact; pin a Git revision for reproduction.